

Financial Universe Control Tower

Azure Data Engineering & Financial Master Data Platform

1. Project Overview

Financial institutions, investment platforms, research firms, and portfolio-management applications depend on accurate and consistent information about financial instruments. However, financial instrument data is often fragmented across asset classes, exchanges, identifiers, classifications, and geographic markets.

The **FinanceDatabase** dataset provides a large financial instrument universe covering equities, ETFs, funds, indices, currencies, cryptocurrencies, and money-market instruments. It also contains attributes such as symbols, names, exchanges, countries, sectors, industries, identifiers, currencies, and classification information.

The objective of this project is to build a **production-oriented Financial Universe Control Tower** using Microsoft Azure that transforms this heterogeneous source into a reliable, versioned, searchable, and analytics-ready **Financial Instrument Master Data Platform**.

The solution must go beyond simple ETL. It should identify duplicate and conflicting instruments, standardize different asset-class schemas, maintain historical classifications, detect changes between source snapshots, measure data quality, and provide business users with an operational view of the financial universe.

2. Business Problem Statement

An investment analytics organization currently receives financial instrument data containing thousands of securities across multiple asset classes and markets.

The organization faces several challenges:

- The same company may have multiple exchange listings.
- Different asset classes have different attributes and schemas.
- Financial instruments may have multiple identifiers such as ISIN, CUSIP, and FIGI.
- Classification information may change over time.
- Duplicate or conflicting identifiers can create incorrect downstream analytics.

- A simple row-count comparison cannot determine whether the financial universe has materially changed.
- Invalid or incomplete records should not be silently discarded.
- Analysts need to distinguish between a **company/entity**, a **financial instrument**, and an **exchange listing**.
- Historical versions of instrument classifications and metadata need to be preserved.
- Business users need a trusted source rather than independently interpreting raw files.

The organization therefore wants to build a centralized **Financial Instrument Master Data Platform**.

The platform should answer:

What financial instruments exist, which canonical entity and listing do they belong to, how are they classified, what identifiers represent them, what has changed since the previous snapshot, and can the organization trust the underlying data?

3. Project Objective

Build an Azure-based data engineering platform that:

1. Ingests financial datasets from the FinanceDatabase repository.
 2. Stores immutable source snapshots in Azure Data Lake Storage Gen2.
 3. Handles heterogeneous schemas across asset classes.
 4. Standardizes the source into a canonical financial instrument model.
 5. Creates a centralized Security/Instrument Master.
 6. Maintains relationships between entities, instruments, listings, and identifiers.
 7. Implements automated data-quality validation.
 8. Separates valid records from quarantined records.
 9. Detects new, removed, modified, reclassified, and conflicting instruments.
 10. Maintains historical versions using Delta Lake and SCD Type 2.
 11. Serves curated data through Azure SQL.
 12. Provides a Power BI-based Financial Universe Control Tower.
-

4. Source Dataset

Use:

FinanceDatabase

Source repository:

<https://github.com/jerbouma/FinanceDatabase>

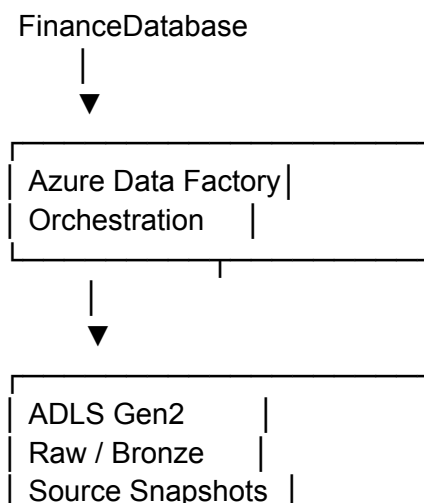
The repository contains financial datasets covering multiple asset classes, including:

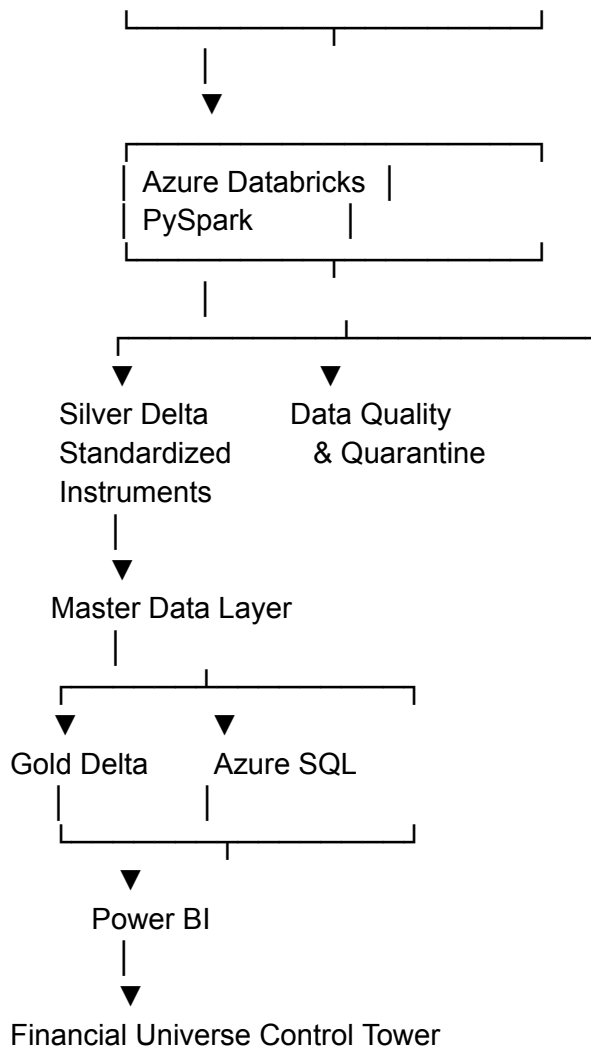
- Equities
- ETFs
- Funds
- Indices
- Currencies
- Cryptocurrencies
- Money markets

The dataset contains different schemas depending on the asset class. Equity-related data can include fields such as sector, industry group, industry, exchange, country, market-cap information, ISIN, CUSIP, FIGI and related identifiers.

The candidate must first perform **source profiling** and document the schema and characteristics of each dataset before designing the ingestion framework.

5. Target Azure Architecture





6. Azure Services to Use

The project must use the following technologies:

Technology	Responsibility
Azure Data Factory	Ingestion and orchestration
Azure Data Lake Storage Gen2	Data lake and source snapshots
Azure Databricks	PySpark transformation and processing

Delta Lake	Lakehouse storage and historical processing
Azure SQL Database	Curated relational serving layer
Power BI	Business and operational analytics

Recommended advanced services

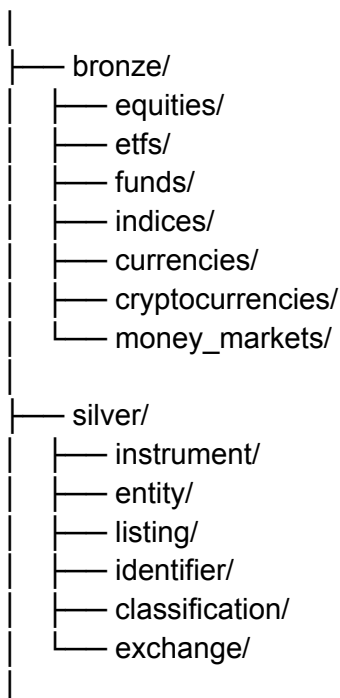
The following may be included as advanced requirements:

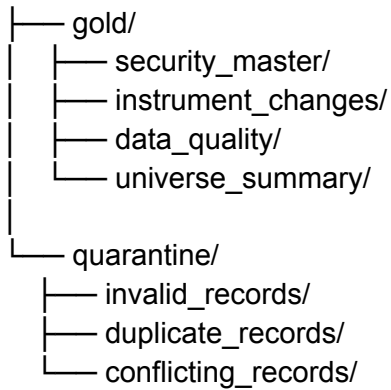
- Azure Key Vault
- Managed Identity
- Azure Monitor
- Azure DevOps / Git
- Microsoft Entra ID
- Azure RBAC

7. Data Lake Architecture

Implement a Medallion Architecture.

ADLS Gen2





8. Bronze Layer

The Bronze layer must preserve the source data with minimal transformation.

Each ingestion should create a source snapshot containing metadata such as:

```
source_file
source_url
source_version
ingestion_timestamp
pipeline_run_id
snapshot_date
```

Do not overwrite previous snapshots.

For example:

```
bronze/equities/
  snapshot_date=2026-08-11/
  snapshot_date=2026-08-12/
  snapshot_date=2026-08-13/
```

This allows the organization to reproduce the state of the financial universe at a specific point in time.

9. Azure Data Factory Requirements

ADF should act as the **orchestration layer**, not the primary transformation engine.

Build a metadata-driven ingestion framework.

Example metadata:

Dataset	Source	Format	Target	Active
Equities	GitHub	CSV	bronze/equities	Y
ETFs	GitHub	CSV	bronze/etfs	Y
Funds	GitHub	CSV	bronze/funds	Y
Indices	GitHub	CSV	bronze/indices	Y
Currencies	GitHub	CSV	bronze/currencies	Y
Crypto	GitHub	CSV	bronze/crypto	Y

The pipeline should dynamically process active datasets.

ADF must demonstrate

- Parameterization
- Metadata-driven ingestion
- Dynamic dataset selection
- Pipeline dependencies
- Retry handling
- Failure handling
- Audit logging
- Trigger-based execution

10. Canonical Financial Instrument Model

Because different asset classes contain different attributes, the candidate must design a canonical model rather than simply unioning all datasets.

dim_instrument

instrument_id
instrument_type
symbol
instrument_name
currency
country
status
created_at
updated_at

dim_entity

entity_id
entity_name
country
entity_type

fact_listing

listing_id
entity_id
instrument_id
symbol
exchange
market
currency
is_primary
effective_from
effective_to

dim_identifier

identifier_id
instrument_id
identifier_type
identifier_value
is_primary
effective_from
effective_to

dim_classification

classification_id
instrument_id
sector
industry_group
industry
effective_from
effective_to
is_current

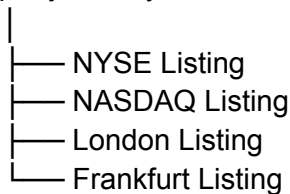
The candidate should justify the model and explain why **entity**, **instrument**, and **listing** should not be treated as the same object.

11. Core Data Engineering Challenge — Entity & Listing Resolution

The same organization can have multiple listings across exchanges.

The platform must distinguish between:

Company / Entity



The solution should therefore identify:

- Canonical entity
- Financial instrument
- Exchange listing
- Primary listing
- Secondary listing
- Country
- Trading currency

The system should avoid treating every ticker as a unique company.

12. Identifier Management

The platform should support multiple identifiers for a financial instrument.

Examples include:

ISIN
CUSIP
FIGI
Composite FIGI
Share Class FIGI

Create an identifier mapping layer.

Example:

instrument_id	identifier_type	identifier_value
10001	ISIN	XXXXXXXX
10001	CUSIP	XXXXXXXX
10001	FIGI	XXXXXXXX

Required validation

The platform should detect cases such as:

Same FIGI

↓

Two different canonical instruments

Such records should be flagged for review rather than silently merged.

13. Data Quality Framework

Implement automated data-quality checks.

Completeness

Check:

- Missing symbol
- Missing instrument name
- Missing exchange
- Missing country
- Missing currency
- Missing identifiers

Uniqueness

Detect:

- Duplicate symbols within the same exchange
- Duplicate identifiers
- Duplicate instrument records

Consistency

Validate:

Identifier → Instrument

Exchange → Country

Currency → Country/Market

Instrument → Asset Type

Referential Integrity

Every reference should resolve to a valid dimension record.

14. Data Quality Scoring

Each instrument should receive a quality score.

Example:

Identifier completeness 25%

Classification completeness 20%

Exchange validity 15%

Country validity	15%
Currency validity	10%
Duplicate risk	15%

Categorize instruments as:

90–100 → Trusted
75–89 → Review
<75 → Quarantine

The scoring methodology should be configurable rather than hard-coded into individual transformation steps.

15. Quarantine Framework

Invalid records must not simply be deleted.

Records failing validation should be moved to a quarantine layer.

Example reasons:

INVALID_IDENTIFIER
DUPLICATE_IDENTIFIER
UNKNOWN_EXCHANGE
UNKNOWN_COUNTRY
MISSING_SYMBOL
CLASSIFICATION_CONFLICT
DUPLICATE_INSTRUMENT

The quarantine dataset should contain:

record_id
source_dataset
source_snapshot
failure_reason
failed_column
original_record
pipeline_run_id
quarantine_timestamp

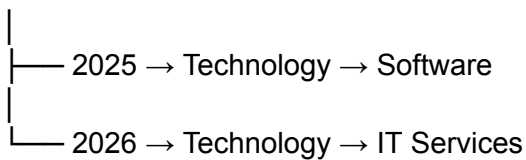
16. Historical Data & SCD Type 2

Financial classifications can change over time.

The platform must preserve classification history.

Example:

Instrument



Implement SCD Type 2 using:

effective_from
effective_to
is_current

Business users should be able to answer:

"What was the classification of this instrument on a particular historical date?"

17. Incremental Processing

The solution must support incremental processing.

Do not assume:

Today's row count = Yesterday's row count

means there was no change.

The system must detect:

NEW
REMOVED
MODIFIED
RECLASSIFIED
RELISTED
IDENTIFIER_CHANGED
DATA_QUALITY_ISSUE

18. Financial Universe Change Detection

This is the **primary unique requirement** of the project.

Create:

fact_instrument_changes

change_id
instrument_id
change_date
change_type
column_changed
old_value
new_value
source_snapshot
pipeline_run_id

Example:

Instrument: 10231

Change Type: RECLASSIFIED

Column: industry

Old Value: Semiconductors

New Value: Semiconductor Equipment

The system should compare two source snapshots and determine exactly what changed.

19. Change Detection Scenario

The following scenario must be supported:

Yesterday
160,000 instruments

Today
160,000 instruments

The system must **not** conclude that there was no change.

It should identify:

2,143 Modified
487 Reclassified
92 Exchange Changes
31 Identifier Changes

The exact numbers will depend on the source snapshots available during implementation.

20. Gold Layer

Create business-ready datasets.

gold_security_master

Contains the current trusted version of each instrument.

gold_instrument_changes

Contains detected changes between snapshots.

gold_data_quality

Contains quality metrics and failures.

gold_universe_summary

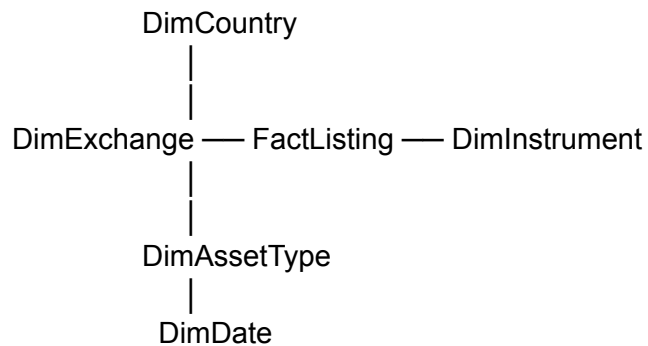
Contains aggregated information by:

- Asset class
 - Country
 - Exchange
 - Sector
 - Industry
 - Currency
-

21. Azure SQL Serving Layer

Expose curated data through Azure SQL.

Use a star schema.



Additional dimensions may include:

DimSector
DimIndustry
DimCurrency
DimIdentifier

Create appropriate:

- Primary keys
- Foreign keys
- Indexes
- Views

- Stored procedures where justified
-

22. Power BI — Financial Universe Control Tower

Build an interactive Power BI solution.

Page 1 — Universe Overview

Display:

Total Instruments
Active Instruments
New Instruments
Removed Instruments
Modified Instruments
Data Quality Score

Visuals:

- Instrument count by asset class
 - Instrument count by country
 - Instrument count by exchange
 - Instrument count by currency
-

Page 2 — Instrument Change Monitor

Display:

New
Removed
Modified
Reclassified
Relisted
Identifier Changed

Include:

- Daily change trend
 - Change type distribution
 - Top affected exchanges
 - Top affected industries
-

Page 3 — Classification Intelligence

Provide drill-down:

Sector
↓
Industry Group
↓
Industry
↓
Country
↓
Exchange

Users should be able to identify how the financial universe is distributed across classifications.

Page 4 — Data Quality Control Tower

Show:

Overall Quality Score
Trusted Instruments
Review Required
Quarantined Records
Identifier Conflicts
Classification Conflicts
Missing Data

Allow users to drill into individual quality issues.

Page 5 — Security Master Explorer

Allow users to search/filter by:

Symbol
Instrument Name
ISIN
CUSIP
FIGI
Country
Exchange
Asset Type
Sector
Industry

The result should show the canonical instrument and its associated identifiers/listings.

23. Advanced Business Questions

The final platform should answer questions such as:

Financial Universe

1. How many unique instruments currently exist?
2. How many unique entities exist?
3. How many exchange listings exist?
4. Which exchanges have the largest instrument universe?
5. Which countries have the largest number of financial instruments?

Change Intelligence

6. How many new instruments appeared this period?
7. Which instruments were removed?
8. Which instruments changed classification?
9. Which exchanges experienced the largest changes?
10. Which identifiers changed between snapshots?

Data Quality

11. Which asset class has the highest data-quality failure rate?

12. Which exchanges generate the most duplicate records?
13. Which identifiers have the highest conflict rate?
14. How many instruments require manual review?

Master Data

15. How many companies have multiple exchange listings?
 16. Which instruments have multiple identifiers?
 17. Which instruments cannot be mapped to a canonical entity?
 18. What was an instrument's classification at a historical point in time?
-

24. Product-Level Requirements

The platform should satisfy the following principles:

Reliability

The same source snapshot should produce the same result when processed repeatedly.

Reproducibility

Historical source states must remain available.

Idempotency

Running the same pipeline multiple times must not create duplicate business records.

Observability

Every pipeline execution should provide:

Run ID
Start Time
End Time
Records Read
Records Written
Records Rejected
Records Modified
Quality Score
Status

Traceability

A business record should be traceable back to:

Source

- Snapshot
 - Pipeline Run
 - Transformation
 - Curated Record
-

25. Security Requirements

Credentials and secrets must not be hard-coded.

Use, where available:

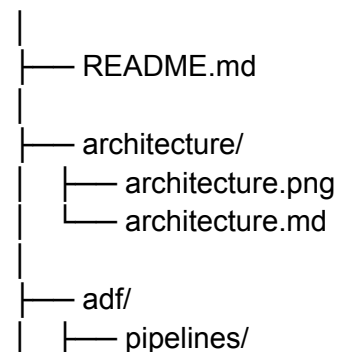
- Azure Key Vault
- Managed Identity
- Azure RBAC
- Secure linked services

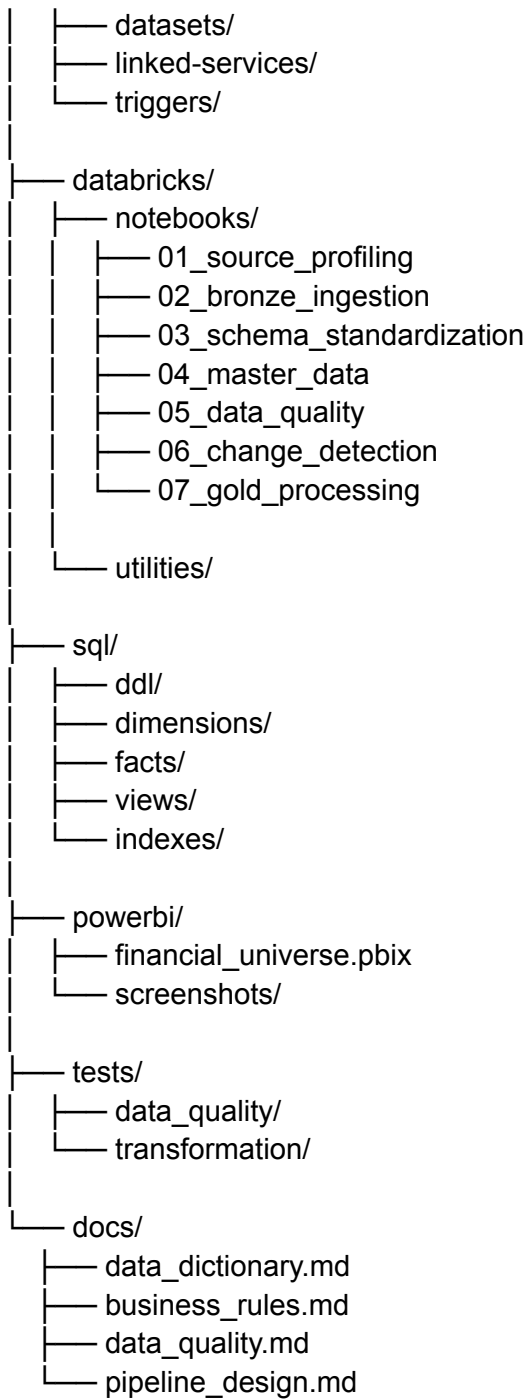
Example of prohibited implementation:

```
storage_key = "xxxxxxx"  
password = "xxxxxxx"
```

26. Recommended Repository Structure

financial-universe-control-tower/





27. Expected Deliverables

The candidate/team must provide:

Azure Data Factory

- Metadata-driven ingestion pipeline
- Parameterized pipelines
- Error handling
- Retry mechanism
- Pipeline audit

ADLS Gen2

- Bronze
- Silver
- Gold
- Quarantine

Azure Databricks

- PySpark notebooks
- Schema standardization
- Data-quality framework
- Master-data processing
- Incremental processing
- Change detection

Delta Lake

- Delta tables
- MERGE/upsert
- SCD Type 2
- Historical snapshots
- Time-travel demonstration

Azure SQL

- Star schema
- DDL
- Indexes
- Curated views
- Serving layer

Power BI

- Financial Universe Control Tower
- Instrument Explorer

- Change Monitor
- Data Quality Dashboard

Documentation

- Architecture diagram
 - Data dictionary
 - Data model
 - Pipeline explanation
 - Data-quality rules
 - Change-detection logic
 - Business rules
 - Setup instructions
-

28. Acceptance Criteria

Area	Acceptance Criteria
Source Analysis	All source asset classes profiled
ADF	Metadata-driven ingestion implemented
ADLS	Immutable source snapshots maintained
Databricks	Heterogeneous schemas standardized
Delta	Bronze/Silver/Gold implemented
Master Data	Canonical instrument model implemented
Listings	Entity/listing relationships maintained
Identifiers	Multiple identifiers supported
Data Quality	Automated validation framework implemented
Quarantine	Invalid records isolated with reasons
SCD	Historical classification maintained
Incremental	New/modified/deleted records detected
Change Detection	Column-level changes identified

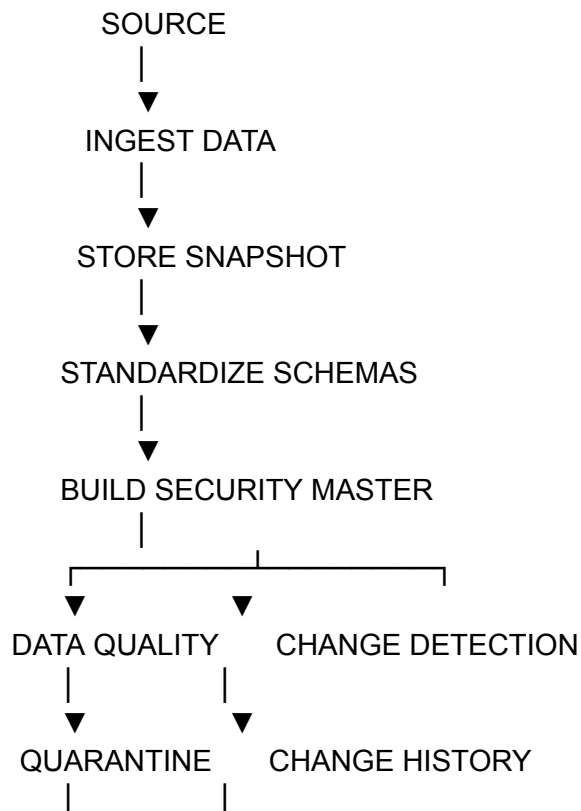
SQL	Curated dimensional model implemented
Power BI	Control Tower implemented
Security	No hard-coded credentials
Audit	Pipeline and data processing audit available
Documentation	Complete technical and business documentation

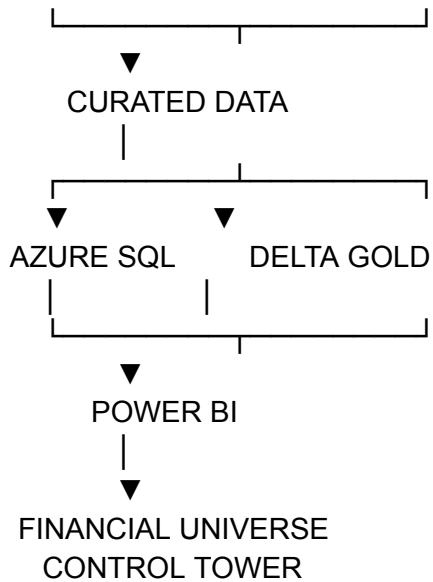
29. Final Product Definition

Financial Universe Control Tower

The completed product should provide a trusted, centralized view of the financial instrument universe.

The core workflow is:





The central engineering question is not:

"How many financial instruments are in the dataset?"

It is:

"Can we reliably determine what financial instruments exist, identify their canonical entities and listings, validate their data, preserve their history, and explain exactly what changed in the financial universe between two source snapshots?"

That is the core challenge that makes this project suitable as an **advanced Data Engineering, Azure, Lakehouse, and Master Data Management assessment**.